정혜원

PROGRAMMER

물리 기반 액션 구현부터,

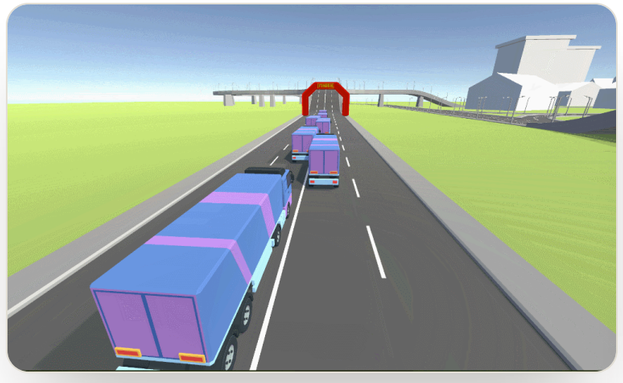
여러 시스템이 얽힌 게임의 상태 관리까지 —

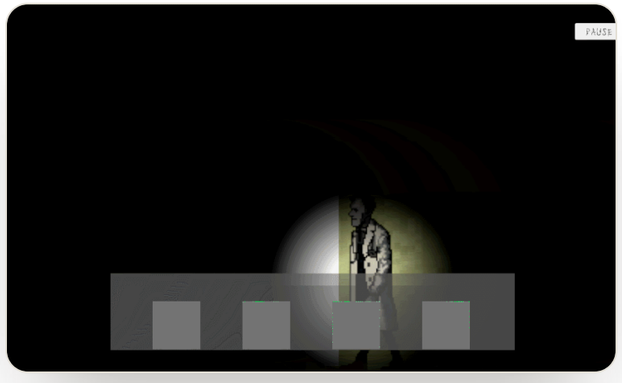
직접 구현하고, 문제가 생기면 끝까지 추적해 해결해왔습니다.

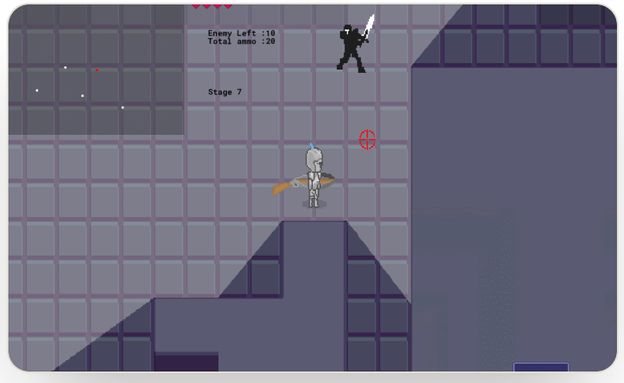
Unity C# Team Project Debugging

GITHUB
github.com/hyewang5617

GAME PORTFOLIO
hyewang5617.github.io/Portfolio







TEAM PROJECT · PROGRAMMER

PH-1

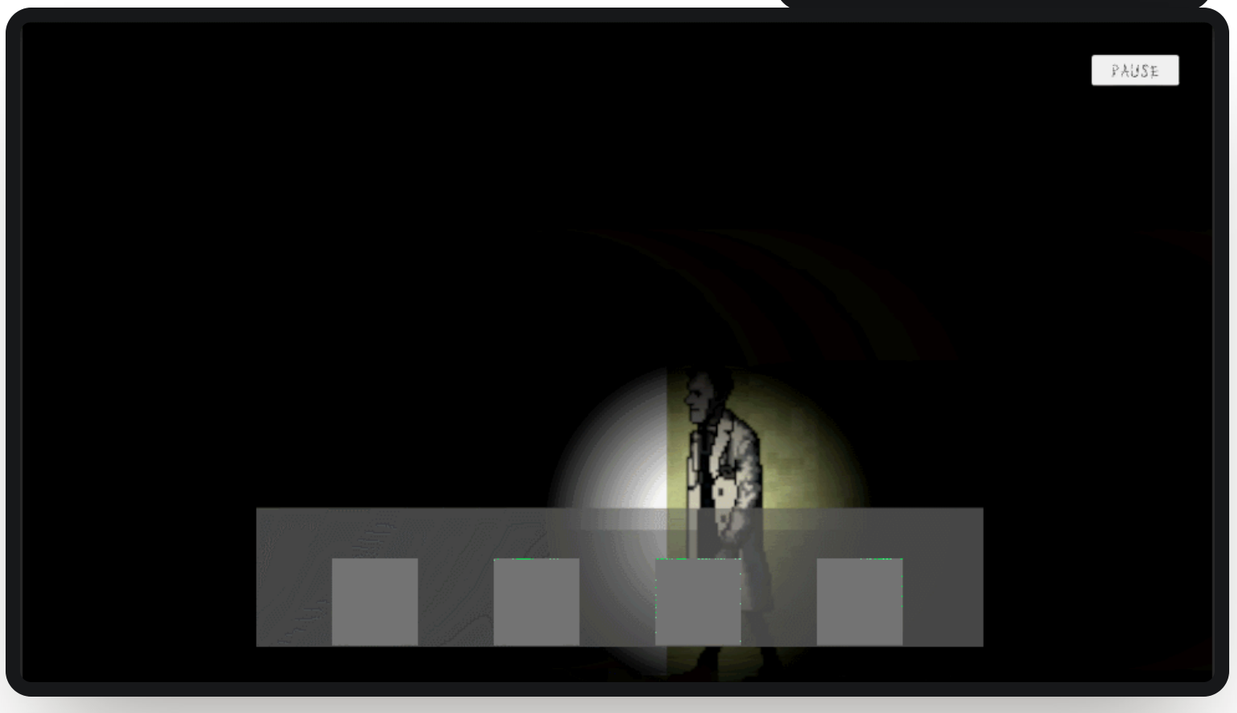
퍼즐 하나를 만드는 것에서 시작해 게임의 '상태'를 관리하는 것까지

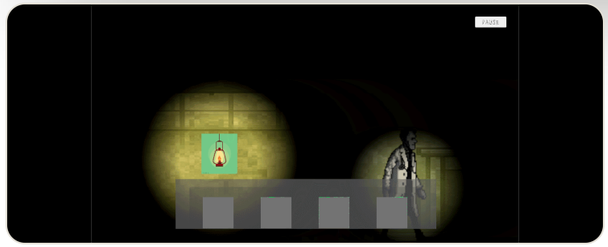
Unity와 C#으로 제작 중인 2D 퍼즐 어드벤처 게임입니다.
플레이어가 공간을 탐색하고 아이템을 사용하며 진행한 이벤트가,
씬을 넘어서도 동일한 진행 상태로 유지되도록 구성했습니다.

SYSTEM FLOW — 독립된 기능이 아니라 하나로 연결된 진행 상태

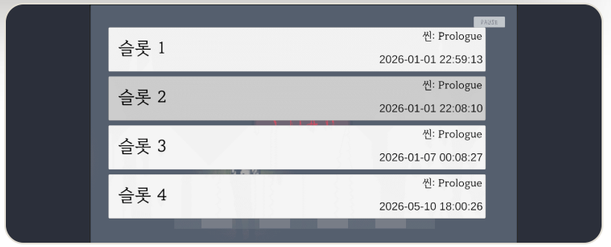


▶ PLAY VIDEO — youtu.be/cwB2SIBhPDU





아이템 획득



세이브 · 로드

</> GameFlag System — VIEW CODE

MY CONTRIBUTION

- Inventory & Item System
- Rhythm Puzzle
- Monster Puzzle
- Save & Load
- GameFlag / Event State
- Spawn & Scene Transition Debugging

TECH

Unity · C# · Git/GitHub

SYSTEM DESIGN

같은 씬인데 조건에 따라 결과가 달랐던 지점에서, 상태가 유지되는 시점을 추적했습니다

MAIN CASE — SCENE TRANSITION DEBUGGING

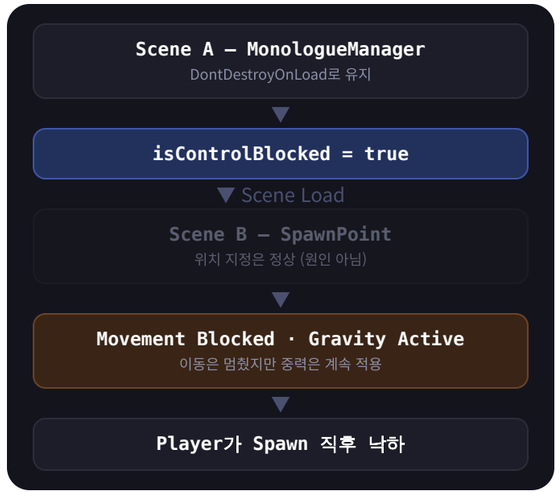
</> VIEW CODE

시네마틱 직후 씬을 이동하면 플레이어가 지정된 문 앞이 아닌 바닥에서 시작했습니다

조건	결과
첫 시네마틱 → 씬 이동	**Player가 바닥으로 떨어짐**
같은 씬 재방문	**정상 Spawn**

처음 의심 — SpawnPoint. 위치 지정 좌표를 확인했지만 Spawn 위치 지정 자체는 정상이었습니다. 재현 조건 차이(첫 방문 vs 재방문)를 근거로 원인을 Scene Lifecycle 쪽으로 옮겨 다시 추적했습니다.

Spawn 코드만 보지 않고 Scene Lifecycle, `DontDestroyOnLoad` 객체, Player State, Rigidbody2D를 함께 추적해 아래 흐름을 확인했습니다.



Door.cs — DoorTrigger.MoveScene()

```
private void MoveScene()
{
    monologueManager.DialogueFinished -= MoveScene;
    ClearSky.Player.isControlBlocked = false;
    thePlayer.pendingEntrance = destinationEntrance;
    StartCoroutine(ChangeSceneWithDelay());
}
```

> Scene Lifecycle과 Player State를 함께 추적해, 씬 전환 이후까지 유지되던 조작 제한 상태를 원인으로 확인했습니다. 씬을 이동시키는 지점에서 이 상태를 명시적으로 해제하도록 수정했습니다.

SECONDARY — GAME STATE ARCHITECTURE

</> GameFlag

이벤트 상태의 소유권을 오브젝트에서 공통 객체로 이동

BEFORE

```
EnemyFindZone → 자체 상태
DoorTrigger   → 자체 상태
Dialogue      → 자체 상태
```

Scene이 재생성되거나 Save/Load를 수행하면 각 오브젝트가 진행 상태를 일관되게 공유하기 어려웠습니다.

AFTER

```
          GameFlag
       /      |      \
EnemyFindZone Door Dialogue
              ↑
          Save Data
```

`GameFlag`가 Runtime Event State와 Save Data를 연결하는 역할을 합니다.

GameFlag.cs

```
public void Set(bool value)
{
    _value = value;
    GameFlags.Set(name, value);
}
public void RestoreFromSave()
{
    _value = GameFlags.GetBool(name, false);
```

> 이벤트 상태의 소유권을 개별 오브젝트에서 공통 상태 객체로 분리해, Scene 재방문과 Save/Load 이후에도 동일한 진행 상태를 참조하도록 구성했습니다.

TEAM PROJECT · 기획 주도 · 개발담당

CRAZY COMMUTER

움직이는 차량을 장애물이 아닌 '발판'으로

게임프로그래밍 수업의 러너 게임 주제에서 출발해, ClusterTruck의
플레이 방식에 착안한 3D 러너·플랫포머 게임을 기획했습니다.
플레이어는 도로를 달리는 차량 위를 뛰어다니며 결승점까지 이동합니다.

2인 팀으로 개발했으며, 콘셉트 기획과 함께 플레이어 조작,
차량 이동·생성 시스템, Stage 1·2, 인게임 UI를 담당했습니다.



VEHICLE SYSTEM — 하나의 발판 시스템

</> VIEW CODE

VehicleSpawner

▼

Spawn Position Validation
겹쳐 생성되지 않는 위치인지 검사

▼

Vehicle Navigation
Look-Ahead Steering으로 코너 대응

▼

Vehicle Movement
플레이어의 발판이 되는 이동 처리

▼

Player-Vehicle Physics
차량 속도 + 입력을 합산해 Lerp 적용

▼

Runtime Recovery
정체·충돌 예외를 실패 조건으로 처리

MY ROLE

Game Concept / Planning · Player Movement · Vehicle System · Vehicle Spawner · Stage 1·2 Level Design · In-game UI

TECH

Unity · C# · Git/GitHub

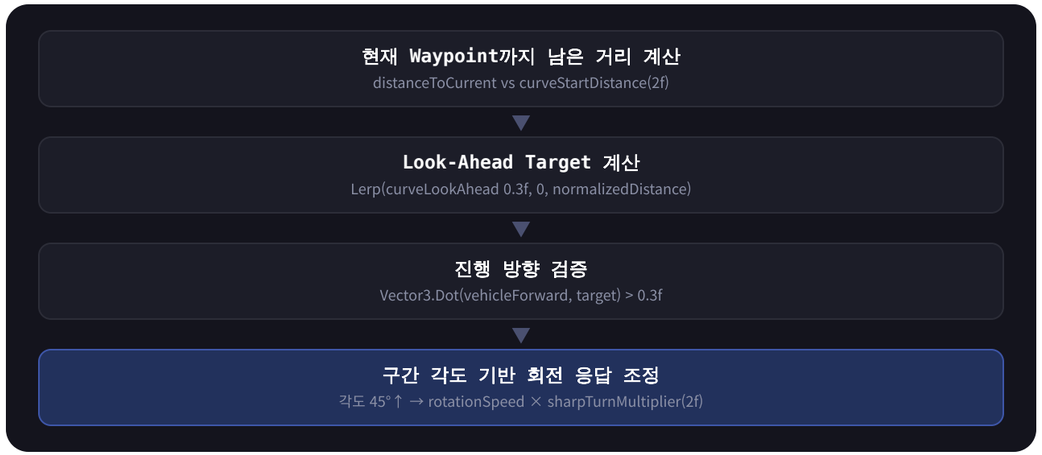
VEHICLE NAV

Waypoint만 순차적으로 추적하면 코너 진입 순간 방향이 급격히 바뀌었습니다

MAIN CASE — LOOK-AHEAD STEERING

</> VIEW CODE

차량이 현재 목표 waypoint만 바라보고 이동할 경우, waypoint가 전환되는 순간 목표 방향이 불연속적으로 꺾여 급커브에서 부자연스러운 회전이 발생하고 차량끼리 충돌하는 경우가 있었습니다.



VehiclePlatform.cs — GetCurvedTargetPosition()

```
float normalizedDistance =
    Mathf.Clamp01(distanceToCurrent / curveStartDistance);
float t = Mathf.Lerp(curveLookAhead, 0f, normalizedDistance);
Vector3 curvedTarget = Vector3.Lerp(
    currentWaypoint.position, nextWaypoint.position, t);
if (Vector3.Dot(vehicleForward, toCurvedTarget) > 0.3f)
    return curvedTarget;
```

VehiclePlatform.cs — FixedUpdate() 회전 응답

```
float angle = Vector3.Angle(currentDir, nextDir);
if (angle > 45f)
    currentRotSpeed = rotationSpeed * sharpTurnMultiplier;

rigid.rotation = Quaternion.Slerp(
    rigid.rotation, targetRotation,
    Time.fixedDeltaTime * currentRotSpeed);
```

> 단순 Waypoint 추적은 코너 진입 시점의 방향 전환을 실패 조건으로 만들었습니다. 남은 거리와 구간 각도를 계산에 포함시켜, 커브의 크기에 따라 회전 응답이 달라지도록 구성했습니다.

TRAFFIC SYSTEM

차량의 자식으로 만든 순간, 두 가지 이동 방식이 같은 프레임에서 부딪혔습니다

MAIN CASE — PLAYER-VEHICLE PHYSICS

</> VIEW CODE

BEFORE

플레이어가 차량에 올라가면 `transform.SetParent(vehicle.transform)`로 차량의 자식으로 만들었습니다 — `Vehicle ㄴ Player Rigidbody`. 하지만 차량의 Transform 이동과 Player Rigidbody의 물리 계산이 같은 프레임에 함께 적용되며 충돌과 이동이 불안정해졌습니다.

AFTER — 부모·자식 관계 제거, 속도를 합산해 매 프레임 보간



VehicleRider.cs — FixedUpdate()

```
Vector3 vehicleVelocity = vehicleRigid.velocity;
Vector3 playerInput =
    player?.GetPlayerInput() ?? Vector3.zero;
Vector3 targetVelocity = new Vector3(
    vehicleVelocity.x + playerInput.x,
    rigid.velocity.y,
    vehicleVelocity.z + playerInput.z);
rigid.velocity = Vector3.Lerp(
    rigid.velocity, targetVelocity,
    Time.fixedDeltaTime * 20f);
```

> Transform 계층과 Rigidbody 물리는 서로 다른 방식으로 위치를 갱신한다는 점을 원인으로 확인했습니다. 부모·자식 관계 대신 차량 속도를 매 프레임 플레이어의 목표 속도에 반영하는 방식으로 바꿨습니다.

SECONDARY — SPAWN VALIDATION

</> VehicleSpawner

```
Collider Bounds 합산 (없으면 Renderer)
        ↓
검사 반경 = vehicleSize × 1.2
        ↓
실패 시 조건 완화 → 최대 60회 재탐색
```

VehicleSpawner.cs

```
checkRadius = vehicleSize * 1.2f;
...
Collider[] overlaps =
    Physics.OverlapSphere(position, checkRadius);
```

RUNTIME RECOVERY

Traffic Jam Recovery

차량 속도가 stalledTimeThreshold(5초) 이상 임계값 아래로 유지되면 실패 조건으로 정의하고 제거했습니다.

Fast Collision Safety

빠른 차량과의 상대 속도가 큰 상황에서 충돌 판정이 누락될 수 있다고 판단해, Player Rigidbody를 Continuous CD·Interpolate로 설정했습니다.

4 STAGES COMPLETED

2인 팀으로 4개 스테이지 모두 플레이 가능한 상태까지 완성

2nd PLACE

수강생 상호 플레이 평가 2위



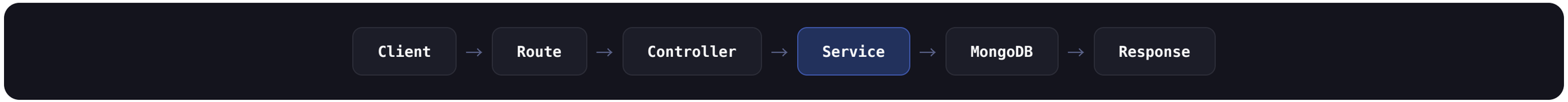
BACKEND DEVELOPMENT EXPERIENCE

Soccer Team Matching Service

Node.js · TypeScript · Express · MongoDB · JWT · Swagger · Docker

Unity 클라이언트 개발과 별도로 참여한 팀 프로젝트로, 축구팀 매칭 서비스의 REST API·인증·배포를 구현했습니다. **클라이언트 요청이 서버 로직과 DB를 거쳐 응답으로 돌아오는 구조**를 직접 구현했습니다.

REQUEST FLOW



MY ROLE

- JWT 기반 로그인 및 인증
- 휴대폰 인증 기능
- 알림 API 및 기존 기능 연동
- 팀 평가 및 평점 기능
- Swagger API 문서화
- 서버 배포 및 트러블슈팅

TECH STACK

Node.js · TypeScript · Express · MongoDB · Mongoose · JWT · Swagger · Docker · Git / GitHub

게임 프로그래머로서 Unity 클라이언트 개발이 중심이지만, 이 프로젝트를 통해 서버·클라이언트가 통신하는 구조를 별도로 구현해 확인했습니다.

01 — AUTHENTICATION & API

인증부터 API까지, 서버 요청 흐름 구현

로그인 시 JWT Access Token과 Refresh Token을 발급하고, 인증이 필요한 API는 미들웨어에서 토큰을 검증하도록 구성했습니다. 만료 토큰과 유효하지 않은 토큰을 구분해 처리했습니다.

PHONE VERIFICATION SMS 인증번호를 bcrypt로 해싱 저장, 5분 만료·재전송 제한·시도 횟수를 관리했습니다.

AUTH FLOW `Client → JWT Middleware → API`

02 — GAME-LIKE SERVICE LOGIC

여러 기능이 연결되는 서비스 로직 구현

NOTIFICATION SYSTEM

알림 Model·Service·Controller·Route를 구현해 팀 가입·매칭 신청·매칭 수락 등 기존 기능과 연결했습니다. 사용자별 조회, 읽지 않은 개수 조회, 읽음 처리 API를 구현했습니다.

TEAM RATING

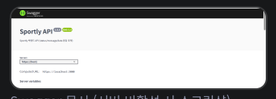
경기 이후 0~5점 평점과 코멘트 기능을 구현했습니다. 자기 팀 평가와 중복 평가를 방지하고, 수정 시 평균 평점이 다시 반영되도록 처리했습니다.

Team Join · Match Apply · Match Accept · Notification · Team Rating

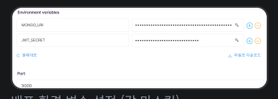
03 — 로컬에서 끝내지 않고 실제 서버 환경까지

</> GitHub PR

개발한 API를 Cloudtype에 배포하고, GitHub 변경 사항이 서버에 반영되도록 배포 과정을 구성했습니다.



Swagger 문서 (서버 비활성 시 스크린샷)



배포 환경 변수 설정 (값 마스킹)



실제 배포 이력 — 실패 후 재배포

PROBLEM 01

Local에서는 Swagger가 동작하지만 배포 서버에서는 접근할 수 없음

서버 실행 방식·배포 환경 확인 → 실행 스크립트를 서버 환경에 맞게 정리 → 배포 환경에서도 Swagger 접근 가능하도록 해결

PROBLEM 02

배포 과정에서 TypeScript Compile 시 메모리 부족 발생

빌드 과정에서 메모리 부족이 발생하는 지점 확인 → 빌드·실행 구조를 조정 → 서버가 정상적으로 실행되도록 해결

FROM BACKEND TO GAME SERVER

TCP 기반 게임 서버를 직접 구현한 경험은 아직 없지만, REST API와 DB를 활용한 백엔드 개발을 통해 클라이언트의 요청이 서버에서 처리되고 데이터와 연결되는 구조를 경험했습니다. Unity 클라이언트 개발 경험과 이러한 서버 개발 경험을 바탕으로, 앞으로는 서버와 클라이언트가 함께 동작하는 게임 구조까지 개발 영역을 넓히고 싶습니다.